

ICS2121, Métodos de Optimización 2025-2

¿Podemos saber si un problema es “fácil” o
“difícil”?

Una breve (e incompleta) introducción a la Teoría
de Complejidad Computacional

Intuición

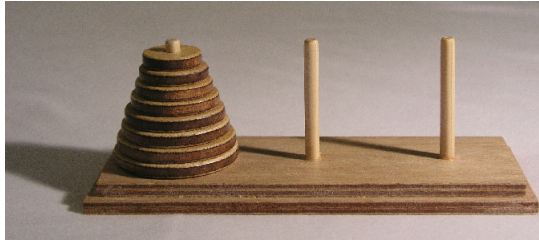
- ¿Qué es más fácil (o difícil)?
 - ▶ Ordenar una lista de números.
 - ▶ Resolver un problema de Programación Lineal.
 - ▶ Resolver un Problema de Vendedor Viajero.
- ¿Cuánto esfuerzo requiere hacer esto?
- ¿Esfuerzo medido en qué?

Cómo se resuelve un problema

- Para resolver un problema usamos un algoritmo.
- Complejidad: la hemos asociado con el número de “iteraciones” y/o el tiempo de computación.
- ¿Cuántas “iteraciones” u “operaciones” se requieren para ordenar n números?

Cómo se resuelve un problema

- ¿Y para este problema? (Torres de Hanoi):



- Objetivo: trasladar los discos desde la barra de la izquierda a la de la derecha, usando la central como ayuda, sin colocar nunca sobre un disco, otro más grande.

Cómo se mide la eficiencia

- Intuición: número de “operaciones” necesarias hasta resolver el problema puede tomarse como una medida de la eficiencia del algoritmo.
- Para comparar dos algoritmos: comparar número de operaciones sobre datos equivalentes, o problemas del mismo “tamaño”.
- Idea: estimar o calcular el número de operaciones como función de algún parámetro relacionado al tamaño del problema.
 - ▶ Cantidad de números.
 - ▶ Número de argollas.
- Cada instancia de un problema tiene un tamaño particular.
- Denotemos por l (del inglés *length*) el tamaño de una instancia de un problema.

Problemas “Fáciles” y “Difíciles”

Definición

*Dado un problema y un algoritmo para resolverlo, diremos que ese **algoritmo resuelve el problema en tiempo polinomial** si el número de operaciones que se requieren para una instancia de tamaño l está acotada por $p(l)$, donde $p()$ es un polinomio.*

- Diremos que un **problema es resoluble en tiempo polinomial** si existe un algoritmo tal que **para cualquier instancia del problema**, la resuelve en tiempo polinomial.
- Entonces: ordenar números es resoluble en tiempo polinomial, las Torres de Hanoi, NO lo es.
- ¿Y Programación Entera...?

Problemas de decisión v/s de optimización

- En el contexto más correcto uno considera problemas de “decisión” más que de optimización.
- Un problema es de decisión si las respuestas posibles son SI o NO.
- Ejemplo, consideremos un poliedro $P = \{x \in \mathbb{R}^n : Ax \leq b\}$ y un vector c .
 - ▶ **Problema de decisión:** ¿Existe $x \in P$?
 - ▶ **Problema de optimización:** $\min\{c^T x : x \in P\}$.
- Si sabemos resolver el problema de decisión, con algunas hipótesis adicionales puedo optimizar.

Clases de Complejidad

Definición

*Se dice que un problema de decisión pertenece a la clase **P** (polinomial) si existe un algoritmo que se detiene con SI o NO en un número de etapas acotado por $p(l)$, donde l es el tamaño de la instancia y p es un polinomio.*

- Estos son los problemas “fáciles”.

Definición

*Se dice que un problema de decisión pertenece a la clase **NP** (no determinístico polinomial) si para una instancia de largo l , dada una **posible** solución al problema, *es posible verificar si la respuesta es SI o NO para esa solución en tiempo polinomial.**

- En otras palabras: es posible verificar si un candidato a solución dado es la respuesta correcta, en tiempo polinomial, pero esto no significa que pueda encontrar una respuesta correcta en tiempo polinomial. (fácil de verificar, no necesariamente fácil de encontrar)

Clases de Complejidad

- Ejemplos:
 - ▶ El Problema de Vendedor Viajero
 - ▶ Programación Entera
 - ▶ El problema de la mochila
- Si el problema es **NP**, ¿puedo dar una cota al **esfuerzo máximo** necesario para encontrar una solución correcta?
- Ilustremos para Programación Entera (decisión):

¿Existe x tal que $Ax \leq b$, $x_j \in \{0, 1\}, j = 1, \dots, n$?

Clases de Complejidad

- Entonces:
 - ▶ Hay problema **fáciles**, en **P**
 - ▶ Hay problemas intratables, que requieren **al menos** tiempo exponencial
 - ▶ Hay problemas que requieren **a lo más**, tiempo exponencial, en **NP**
- ¿Será posible que pueda resolver un problema **NP** de manera fácil?
- La práctica parece indicar lo contrario.

Los problemas NP-Completo

- Existe un subconjunto de los problemas en **NP** que son los más difíciles de entre todos.

Definición

Sean A y B dos problemas de decisión. A es *polinomialmente reducible* a B si existe una transformación g *evaluable en tiempo polinomial* que transforma una instancia e de A en una instancia $e' = g(e)$ de B y e es SI si y sólo si la respuesta de $g(e)$ es SI.

Definición

Un problema B en la clase **NP** se dice **NP-Completo** si cualquier problema A en **NP** se puede reducir polinomialmente a B .

- ¿Qué pasa si descubro un algoritmo polinomial para un problema **NP-completo**?

Los problemas NP-Completo

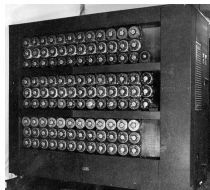
- Se conocen muchos problemas **NP-completos**:
 - ▶ El problema del vendedor viajero.
 - ▶ Corte máximo en un grafo (MAXCUT).
 - ▶ Conjunto Independiente en un grafo.
 - ▶ El problema de la mochila.
 - ▶ Programación entera en muchas de sus variantes.
 - ▶ Un “montón” de los que aparecen en aplicaciones...
- El primer problema **NP-completo** fue identificado por Cook [1972]. (SATISFABILIDAD).
- Después Held y Karp [1972] identificaron otros.
- De ahí en adelante muchos otros fueron identificados...
- Esto explica por qué a veces se observa tanto estudio de los problemas que son **NP-completos**.

Los problemas NP-Completo

- **Pregunta:** dado que $P \subset NP$, ¿qué será cierto: $P = NP$ ó $P \neq NP$?
- Si fuera $P = NP$, entonces significa que efectivamente existen algoritmos eficientes para problemas enteros y combinatoriales, **aunque hasta ahora no los hemos encontrado**.
- Si la inclusión es estricta, entonces los problemas combinatoriales son, efectivamente, muy difíciles.
- La pregunta anterior aún está abierta.
(Y vale algunos millones:
<http://http://www.claymath.org/millennium-problems>)
- Algunos piensan que podría ser un caso de problema “indecidable”

Nota sobre la verdadera teoría formal

- Hemos hablado de “tamaño” de problemas y “operaciones” de un algoritmo y pensamos en: número de variables y similares, sumas, multiplicaciones, comparaciones...
- La teoría formal es un poco más complicada que eso...
- Un algoritmo es un “programa” en un modelo formal de un computador.
- Este modelo formal es la Máquina de Turing. (por Alan Turing).



- El concepto de “tamaño” tiene un significado más preciso: es el número de símbolos necesarios para codificar el problema en el lenguaje adecuado: en este caso, número de bits (incluyendo signos y marcadores).

La complejidad de Programación Lineal

- Los problemas de Programación Lineal se resuelven habitualmente usando el método SIMPLEX.
- Recordemos el resultado de 1972 de V. Klee y G. Minty:
- Instancia de problema de programación lineal en n variables para la cual el método SIMPLEX (con varios criterios conocidos de selección de variables) requiere realizar 2^n pivotes, luego, **no es un algoritmo de tiempo polinomial**.
- La pregunta sobre la complejidad de P.L. permaneció abierta hasta 1979:
- Leonid Khachiyan presentó un algoritmo aplicable a P.L. que corría en tiempo polinomial:
 - ▶ Método de las Elipsoides,
 - ▶ origen anterior: Shor , Nemirovski y Yudin en los 70.

La complejidad de Programación Lineal

- Pero en la práctica, el Método de las Elipsoides era horrible de lento!
- Y el SIMPLEX siguió reinando...
- Aún es una pregunta abierta entender por qué el método SIMPLEX, pese a ser ineficiente en el peor caso, se comporta muy bien “en promedio”.
- Un comportamiento “promedio”: $O(\max\{m, n\})$ pivotes para llegar al óptimo.
- Pero la última palabra en polinomialidad para P.L. no estaba dicha...
- 1984: N. Karmarkar presentó un nuevo algoritmo que también era polinomial para programación lineal.
- El método presentado era un **Algoritmo de Punto Interior**.
- Estos proveen un método “teórico” polinomial para resolver P.L. pero, además, son eficientes en la práctica...
- Los estudiaremos un poco más adelante en el curso.